# Topic 7: Transport, Session, Presentation & Application Layers

Course outline weeks 8-11: transport design issues, connection management, session layer & RPC, presentation layer & compression, application layer (file transfer, email, virtual terminals) + lab commands. TCP vs UDP is in 4 of 5 papers.

## How often this topic is examined

| Question asked | Paper(s) | Marks |
|---|---|---|
| TCP vs UDP | 2018 Q2a [5], SE2024 Q5e [4] | 4-5 |
| How data is sent with UDP (connectionless) | 2023 Q5c [4], ISA2024 Q5a [4] | 4 |
| Why TCP is called reliable (with example) | 2018 Q5a | 6 |
| TCP 3-way handshake with diagrams | 2018 Q3c | 6 |
| How DNS works - recursive lookups | 2018 Q2b | 10 |
| Troubleshooting a PC that can't reach the Internet | 2018 Q2c | 10 |
| Layer 4 functions | 2025 Q1a.ii | (see Topic 2) |
| Outline topics: connection management, RPC, compression, FTP, email, virtual terminals | Course outline wks 8-11 | likely |

## 1. TCP vs UDP

**EXAM QUESTION** 2018 Q2a [5], SE2024 Q5e [4] **[4-5]**
Differentiate between TCP and UDP as they apply to layer four (4) of the OSI model. / Discuss the differences between TCP and UDP.

| Feature | TCP (Transmission Control Protocol) | UDP (User Datagram Protocol) |
|---|---|---|
| Connection | **Connection-oriented** - 3-way handshake before data, 4-way close | **Connectionless** - just sends datagrams, no setup |
| Reliability | **Reliable** - sequence numbers, ACKs, retransmission of lost segments | **Unreliable** - no ACKs, no retransmission (best effort) |
| Ordering | Delivers data **in order** (reorders using sequence numbers) | No ordering - datagrams may arrive out of order |
| Flow & congestion control | Yes - sliding window, slow start | None |
| Error checking | Checksum + recovery | Checksum only (optional in IPv4); damaged datagram discarded |
| Header size | **20-60 bytes** | **8 bytes** (source port, destination port, length, checksum) |
| Speed / overhead | Slower, more overhead | **Faster**, low overhead, low latency |
| Data unit | Segment (byte stream) | Datagram (message-oriented) |
| Broadcast/multicast | No (unicast only) | Supports broadcast & multicast |


Made by King Tine, the AI Engineer

| Feature | TCP (Transmission Control Protocol) | UDP (User Datagram Protocol) |
|---|---|---|
| Applications (ports) | HTTP 80, HTTPS 443, FTP 20/21, SMTP 25, SSH 22, Telnet 23, POP3 110 | DNS 53, DHCP 67/68, SNMP 161, TFTP 69, VoIP/RTP, online gaming, video streaming |

## 2. UDP data transmission

**EXAM QUESTION** 2023 Q5c, ISA2024 Q5a **[4]**
In contrast to TCP, UDP is described as a connectionless protocol. Briefly explain how data is transmitted between two computers using the UDP protocol.

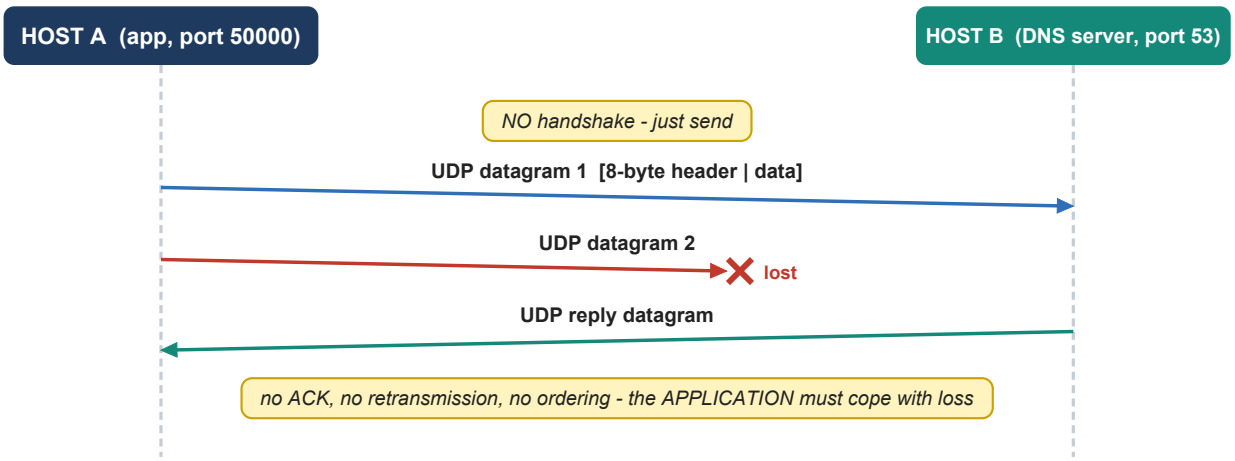
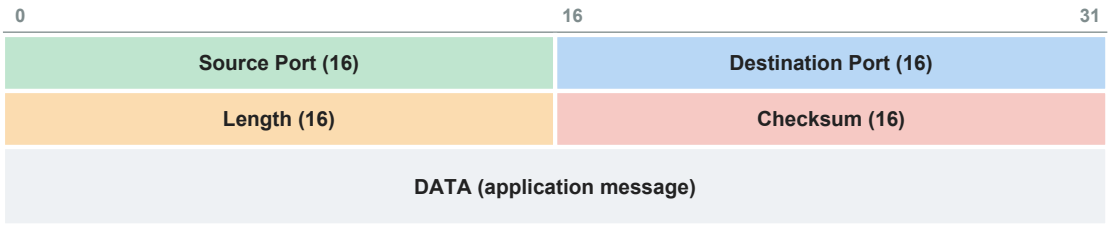



*Figure: UDP: connectionless transmission*




*UDP header = only 8 bytes (TCP header = 20-60 bytes)*

*Figure: the 8-byte UDP header*

1. The application passes a message to UDP; UDP adds an **8-byte header** (source port, destination port, length, checksum) creating a **user datagram**.
2. There is **no connection set-up** (no handshake); the datagram is handed straight to **IP**, which routes it independently to the destination IP address.
3. Each datagram is **independent** - they may take different paths, arrive **out of order, duplicated or not at all**.
4. The receiver's UDP checks the **checksum**; if corrupted the datagram is **silently discarded**; otherwise it is delivered to the application identified by the **destination port**.
5. **No acknowledgements, no retransmission, no flow or congestion control** - any reliability must be handled by the application. This makes UDP **fast with low overhead**, ideal for DNS queries, DHCP, VoIP, live video, gaming.

## 3. Why TCP is reliable

**EXAM QUESTION** 2018 Q5a **[6]**
With the aid of an example explain why TCP is called a reliable protocol.

- **Connection establishment** - 3-way handshake makes sure both sides are ready and agree initial sequence numbers.
- **Sequence numbers** - every byte is numbered, so the receiver can put segments **in order** and detect **duplicates/missing** data.
- **Acknowledgements (ACK)** - the receiver confirms what it received (cumulative ACK = next byte expected).
- **Retransmission** - if no ACK arrives before the **retransmission timer** expires (or 3 duplicate ACKs arrive), the sender resends the segment.
- **Checksum** - detects corrupted segments, which are discarded and later retransmitted.
- **Flow control** (sliding window - receiver advertises its window) and **congestion control** stop data being lost through overflow.

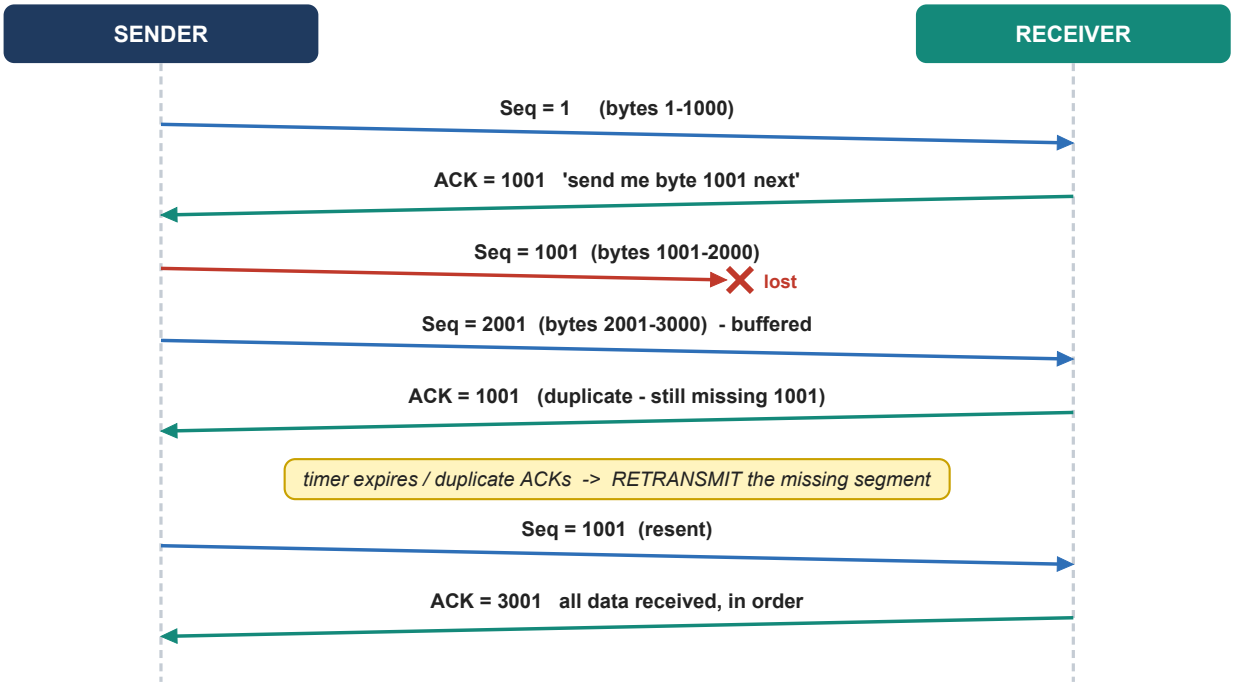



*Figure: how TCP recovers a lost segment*

## 4. TCP 3-way handshake & connection management

**EXAM QUESTION** 2018 Q3c **[6]**
Describe the TCP 3-way handshake using diagrams.

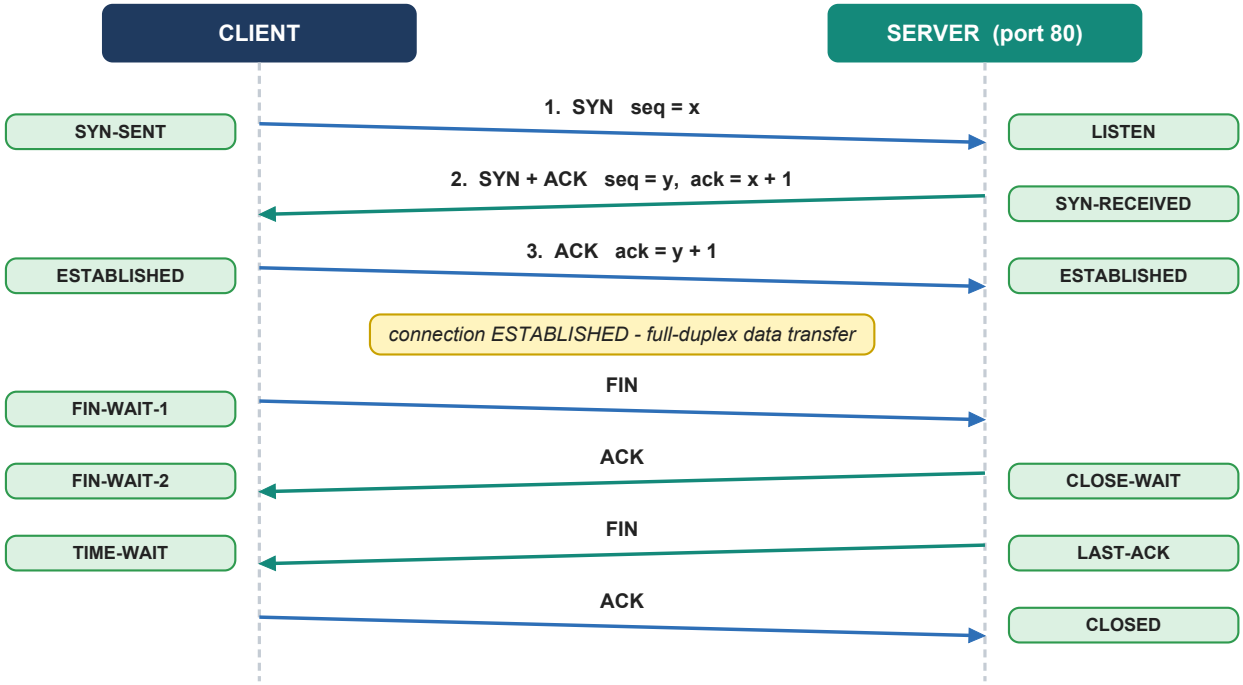



*Figure: TCP 3-way handshake and 4-way close with connection states*

1. **SYN** - the client sends a segment with the **SYN flag** set and its **initial sequence number (ISN = x)**, requesting a connection.
2. **SYN-ACK** - the server replies with **SYN and ACK** flags: its own ISN (y) and acknowledgement number **x + 1** (it accepts and is ready).
3. **ACK** - the client sends an **ACK** with ack number **y + 1**. The connection is now **ESTABLISHED** and full-duplex data transfer starts.

Purpose: synchronise sequence numbers in both directions, confirm both sides are reachable and ready, agree parameters (MSS, window size). **SYN flooding** attack abuses step 2 (half-open connections).

**Transport design issues (outline):** addressing (ports/sockets = IP + port), connection establishment & release, flow control & buffering, multiplexing many connections over one network connection, crash recovery, error control.

## 5. DNS

> **EXAM QUESTION** 2018 Q2b **[10]**
> Describe in detail how DNS works with particular focus on recursive lookups.

**DNS (Domain Name System)** is a distributed, hierarchical database that translates human-readable **domain names** (www.hit.ac.zw) into **IP addresses** (and vice versa). It uses **UDP port 53** (TCP 53 for zone transfers/large replies).

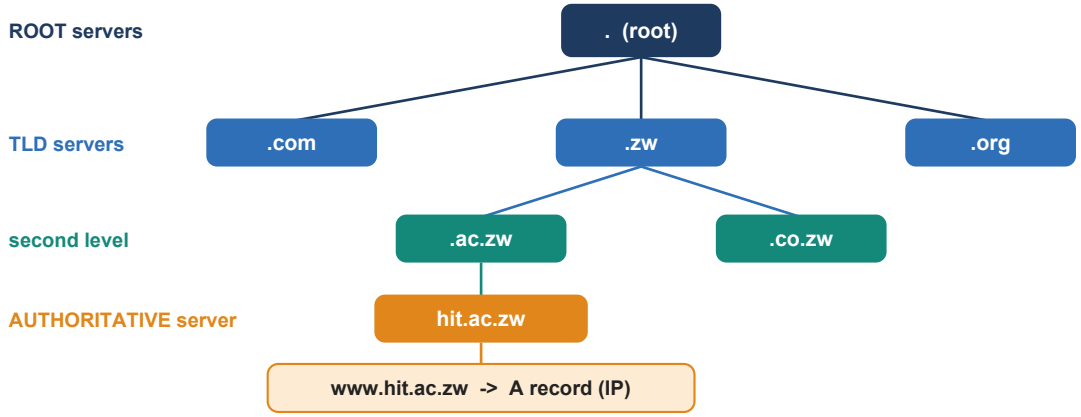



*Figure: the DNS hierarchy*

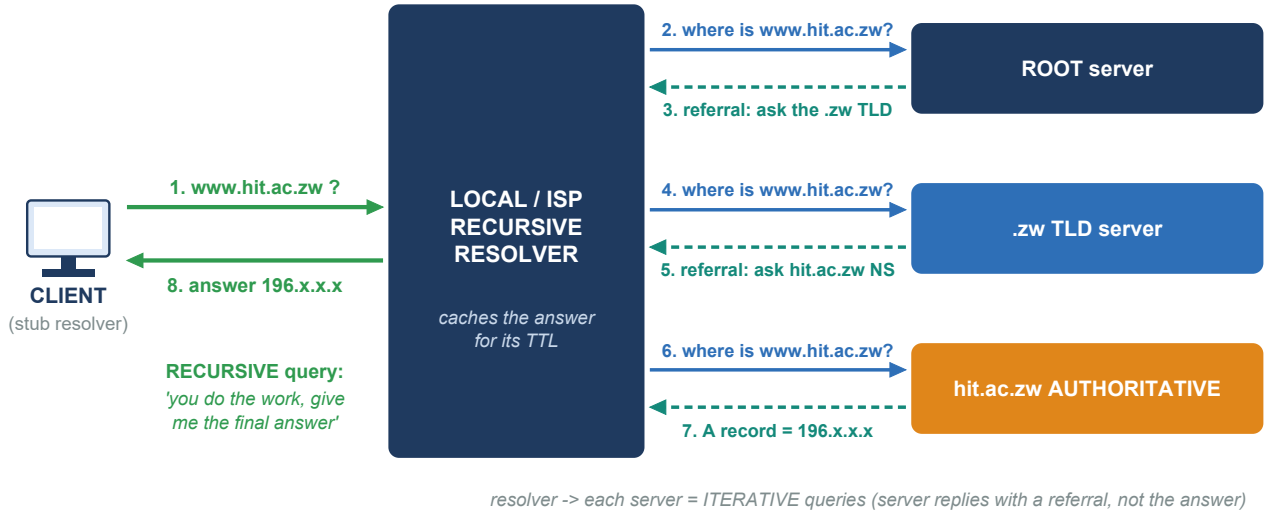



*Figure: a recursive DNS lookup step by step*

1. The user types www.hit.ac.zw. The browser/OS first checks its **local cache** and the **hosts file**.
2. If not found, the **stub resolver** sends a **recursive query** to the configured DNS server (ISP or company resolver). 'Recursive' means: *'give me the final answer or an error - you do all the work'*.
3. The resolver checks its cache. If not cached, it queries a **root server**, which refers it to the **.zw TLD server**.
4. The resolver asks the **TLD server**, which refers it to the **authoritative name server** for hit.ac.zw.
5. The resolver asks the **authoritative server**, which returns the **A record** (IPv4) / AAAA (IPv6) for www.hit.ac.zw.
6. The resolver **caches** the answer (for its TTL) and returns it to the client, which then connects to the web server's IP.
7. Note: steps 3-5 between resolver and other servers are **iterative queries** (each server returns a referral). Only the client-to-resolver query is **recursive**. In a fully recursive scheme each server would itself query the next one and pass the answer back down.

**Record types:** A (IPv4), AAAA (IPv6), CNAME (alias), MX (mail server), NS (name server), PTR (reverse lookup), SOA. **Security:** DNS spoofing/cache poisoning -> DNSSEC.

## 6. Troubleshooting 'no Internet'

**EXAM QUESTION** 2018 Q2c **[10]**
A co-worker calls the help desk complaining about her PC not going to the internet. Outline the procedure you would take in troubleshooting this problem.

Use a structured **bottom-up OSI approach** (and first ask questions: what changed? is it just her or everybody? any error message?).

1. **Physical (L1):** is the cable plugged in, NIC link lights on, Wi-Fi switched on/connected to the right SSID? Try another cable/port.
2. **Check IP configuration:** run `ipconfig /all`. Does the PC have a valid IP, mask, gateway and DNS? A **169.254.x.x** (APIPA) address means DHCP failed -> `ipconfig /release` then `ipconfig /renew`; check the DHCP server.
3. **Test the TCP/IP stack:** `ping 127.0.0.1` (loopback) - fails = reinstall/reset TCP/IP (`netsh int ip reset`) or NIC driver.
4. **Test the local network:** `ping` own IP, then **`ping` the default gateway** - fails = LAN/switch/VLAN problem.
5. **Test beyond the gateway:** `ping 8.8.8.8` - fails = router/ISP/firewall problem; use `tracert 8.8.8.8` to see where packets stop.
6. **Test DNS:** `ping www.google.com` or `nslookup www.google.com` - IP works but names fail = DNS problem -> fix DNS server settings, `ipconfig /flushdns`.
7. **Application level:** check browser proxy settings, firewall/antivirus blocking, malware, try another browser.
8. **Compare** with a colleague's PC; if many users are affected escalate to network team/ISP. **Document** the problem and solution, confirm with the user that it works.

### Lab commands from the course outline (practical 6)

| Command | What it does |
|---|---|
| ipconfig / ipconfig /all | Shows IP address, mask, gateway (+ MAC, DHCP, DNS with /all) |
| hostname | Displays the computer's name |
| ping <ip> | Sends ICMP Echo Requests to test reachability and round-trip time |
| tracert <ip> | Lists every router (hop) on the path to the destination (uses TTL) |
| netstat -an | Shows active TCP/UDP connections and listening ports |
| nslookup <name> | Queries DNS for a name's IP address |
| arp -a | Shows the ARP cache (IP -> MAC mappings) |

**UTP cables (practical 4): straight-through** (same standard both ends, T568B-T568B) joins **unlike** devices: PC-switch, switch-router. **Crossover** (T568A one end, T568B the other) joins **like** devices: PC-PC, switch-switch, router-router (or PC-router). T568B order: white-orange, orange, white-green, blue, white-blue, green, white-brown, brown.

## 7. Session & presentation layers (outline weeks 8-9)

- **Session layer design issues:** dialog control (who talks when, half/full duplex), **synchronisation** (checkpoints so a 100 MB transfer that fails at 90 MB restarts from the last checkpoint), session establishment/termination, token management.
- **Remote Procedure Call (RPC):** lets a program **call a procedure on a remote machine as if it were local**. The client calls a local **client stub**, which **marshals** (packs) the parameters into a message and

sends it; the **server stub** unpacks them, calls the real procedure and returns the result the same way. Hides network details from the programmer (e.g. gRPC, Java RMI, NFS).

- **Presentation layer design issues:** translation between data formats (ASCII/Unicode/EBCDIC, big/little endian, ASN.1), **encryption** for privacy, **compression** to reduce the number of bits.
- **Data compression techniques: Lossless** - original recovered exactly: **Run-Length Encoding** (AAAAAABB -> 6A2B), **Huffman coding** (shorter codes for frequent symbols), **LZW/LZ77** (dictionary - ZIP, GIF, PNG). **Lossy** - some detail discarded for much smaller files: **JPEG** (images, DCT), **MPEG/MP4** (video), **MP3** (audio).

## 8. Application layer services (outline weeks 10-11)

| Service | Protocols / how it works |
|---|---|
| File transfer, access & management | **FTP** - TCP port 21 control connection + port 20 data connection; commands USER, PASS, GET/RETR, PUT/STOR; active vs passive mode. **TFTP** (UDP 69, simple, no login). **SFTP/FTPS** (secure). FTAM (OSI File Transfer, Access and Management). |
| Electronic mail | **SMTP** (TCP 25/587) - sends/pushes mail from client to server and between servers. **POP3** (110) - downloads mail and usually deletes from server. **IMAP** (143) - keeps mail on server, syncs folders across devices. **MIME** - attachments & non-ASCII content. Components: user agent (Outlook), mail transfer agent, mailbox/mail server |
| Virtual terminals / remote login | **Telnet** (TCP 23) - remote command-line login, plain text (insecure). **SSH** (TCP 22) - encrypted remote login. Network Virtual Terminal = a standard terminal so any terminal type can use any host |
| Web | **HTTP** (80)/**HTTPS** (443) - request/response (GET, POST); URLs; HTML |
| Network management | **SNMP** (UDP 161/162) - manager polls agents on devices; MIB |

**REMEMBER:** TCP = phone call (connect, reliable, in order). UDP = SMS/post (just send, fast, no guarantee). Handshake: SYN -> SYN-ACK -> ACK. DNS: client->resolver recursive, resolver->root->TLD->authoritative iterative. Troubleshoot bottom-up: cable -> ipconfig -> ping 127.0.0.1 -> ping gateway -> ping 8.8.8.8 -> nslookup.